KOIOS · YOUR COMPUTERS · COMPANION 0.20.0

A folder of one computer, a disk on another

Any computer signed in to your account can attach a folder of another as a disk of its own — on a Mac, on Windows, on Linux — through the Companions and their Cloudflare tunnels. Files come over as they are opened. What was built, why, how it was checked, and how to try it.

Branch claude/epic-darwin-18jc7s at 2b471f6 · 2026-10-20 · printed from docs/shared-folders.md and docs/shared-folders-process.md

Part 1 · What it is, and how to use it

- 1. Attaching a folder
- 2. Where it appears, on each system
- 3. What is kept on the other computer
- 4. How it reaches the other computer
- 5. What the key opens
- 6. When the other computer sleeps
- 7. Reference

Part 2 · How it was built, checked and measured

- 1. The ask
- 2. The choices
- 3. How it fits together
- 4. The order of work, and what each run found
- 5. The tests
- 6. What isn't verified yet
- 7. Trying it, step by step

PART 1

What it is, and how to use it

Any computer signed in to your Koios account can attach a folder of another one as a disk of its own. A Mac can attach a folder of a Windows PC, a Linux box a folder of a Mac, and so on, in any direction. The folder shows up where the system shows disks: in Finder, in File Explorer, or in your file manager. Every program there can open it, the way it would open a network drive. Files stay on the computer they are on. They come over when they are opened, and what is written on the other side goes back when it is closed.

How it was built and checked, with a step-by-step test plan, is in shared-folders-process.md. Both documents are printed as one PDF, `docs/shared-folders-process.pdf` (`node scripts/compute-mount-pdf.mjs shared-folders`).

This needs Koios Companion **0.20.0 or later** on both computers, each signed in to the same account (Settings → Updates).

---

1. Attaching a folder

Settings → Compute & playgrounds → Folders across your computers:

1. **From**: the computer the folder is on, and the folder. Browse to it or type its path.
2. **To**: the computer it goes to.
3. **Keep here**: how much of it is kept on that computer (section 3).
4. **Attach on ...**

The same card lists what each computer has attached, with **Detach**, and what each one shares, with **Stop sharing**. From a terminal:

```
# on the computer with the folder
reader-companion share ~/Documents/Thesis          # prints: reader-companion attach koios-
attach:…
reader-companion share                             # what is shared here
reader-companion share --stop ~/Documents/Thesis   # every computer that had it attached is
cut off

# on the other computer
reader-companion attach koios-attach:…             # --cache used|all|none  --backend
auto|fuse|webdav  --at <where>
reader-companion attach                            # what is attached here
reader-companion attach --detach <id>
reader-companion attach --install-driver           # FUSE-T on a Mac, WinFsp on Windows
```

The code is the folder's address and its key in one word. Keep it to yourself: anyone who has it can open that folder, and nothing else, until it is stopped.

---

2. Where it appears, on each system

The folder is mounted in one of two ways. The Companion picks the faster one that the computer has:

	WITH A DRIVER (FUSE)	WITHOUT ONE (WEBDAV, THE DEFAULT UNTIL A DRIVER IS INSTALLED)
Mac	FUSE-T (or macFUSE), at `~/Koios Drives/<name>` and in Finder's sidebar	The Mac's own network-drive client (`mount_webdav`), at `~/Koios Drives/<name>`
Windows	WinFsp, as a drive letter (from `Z:` down)	A mapped network drive (`net use`), as a drive letter. Needs the WebClient service, which Windows has on by default
Linux	libfuse 2, at `~/Koios Drives/<name>`	GNOME's `gio`, under `/run/user/<uid>/gvfs`, with a link at `~/Koios Drives/<name>`

Nothing is installed without you. On a Mac or a Windows PC without a driver, the card offers **Install FUSE-T** or **Install WinFsp**. That downloads the driver's own installer from its GitHub releases and opens it, and the system asks for your password (or UAC) as for any installer. On Linux it gives the one command to run (`sudo apt install libfuse2`). Attach again afterwards to use the driver.

Why offer the driver at all: the system's network-drive clients are slower and fussier. Windows' WebClient won't take a file over **50 MB** unless the limit is raised in the registry. Finder copies a whole file before it opens it. With a driver, a program reads just the part of a file it asks for.

3. What is kept on the other computer

KEEP HERE	WHAT IT DOES	GOOD FOR
Only what's used (default)	A folder is listed when it is opened. A file comes over in 1 MB blocks as they are read, and is kept until it changes.	Big folders where you open a few files.
Everything, in the background	The same, and then the rest of the folder comes over, smallest files first, up to 4 GB.	Working while the other computer sleeps or travels.
Nothing kept	Every read goes to the other computer.	A folder that changes all the time, or a computer short of disk.

The cache is under `~/.reader-companion/attached/<id>/`, and **Detach** removes it.

4. How it reaches the other computer

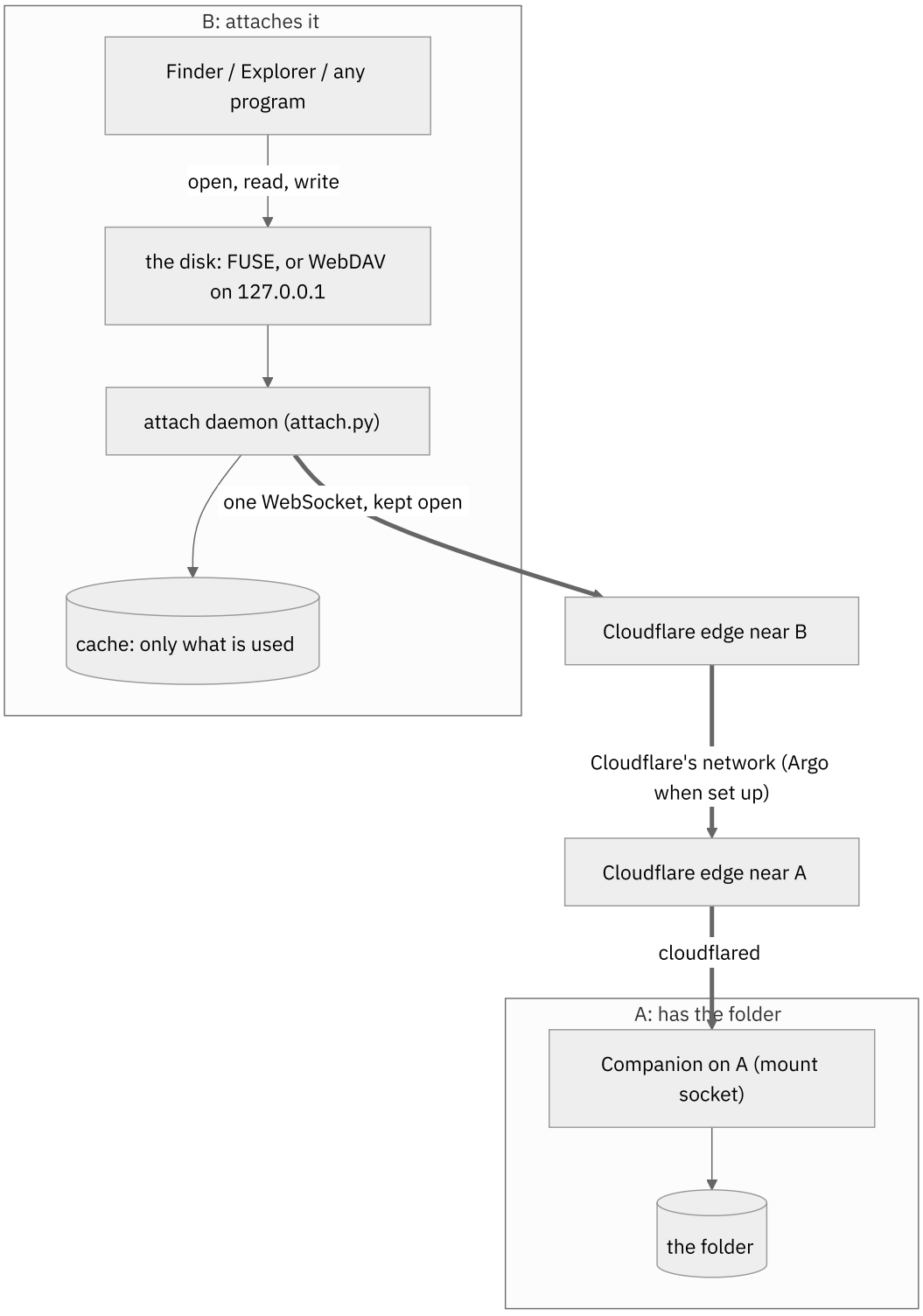


Figure 1. The attaching computer dials the other computer's Companion through its Cloudflare tunnel: the same socket the project mount uses (compute-mount.md), with the same binary frames. The page only introduces the two computers to each other. No file passes through it, or through the Worker.

- **One socket, kept open.** The daemon holds one WebSocket to the other Companion and reconnects with backoff if it drops.

- **Listed when opened.** Unlike a project mount, a share sends no tree up front, so a folder with a million files costs nothing to attach. Each folder is listed the first time it is opened (op `list`). The Companion then watches the folders that have been listed, and pushes their changes (op `entries`) as they happen.
- **Read in blocks.** A read fetches the 1 MB blocks it touches, up to 4 MB per request. Blocks are kept, and a file is whole once every block is there.
- **Written on close.** A file written here goes back when the program closes it, in 4 MB pieces. The Companion writes it beside the original and swaps it in, so nobody there ever sees half a file.
- **The system's own files stay put.** `.DS_Store`, `._*`, `Thumbs.db`, `desktop.ini`, `.Spotlight-V100` and the like are kept on the attaching computer. They are never written into your folder.

5. What the key opens

- Each shared folder has a key of its own, kept in `~/.reader-companion/shares/` (0600). It is never the Companion's token, and the list of shares never shows it.
- The socket refuses any path outside the folder, including a link that leads out of it.
- Neither the whole disk nor your whole home folder can be shared. Pick a folder inside it.
- **Stop sharing** forgets the key and closes every socket that used it, at once.
- On the attaching computer, the key is in `~/.reader-companion/attached/<id>/attach.json` (0600). It is never on a command line, where other users could read it. The WebDAV server listens on 127.0.0.1 only, under a random path, so another program on that computer can't find it by trying ports.

6. When the other computer sleeps

- What has been read stays readable. A folder listed before reads from its last listing, after one short try.
- A read of something not kept waits up to 15 minutes for the other computer to come back, then fails with an I/O error. **Everything, in the background** avoids this.
- Writes wait. A file written while the other computer is away is kept, and the program's `close()` succeeds. It is sent when the socket is back. The queue is on disk, so it survives a restart.
- A computer with a folder attached somewhere is kept awake while it is attached, as for the project mount: `caffeinate`, `systemd-inhibit` or `SetThreadExecutionState`. A laptop with its lid closed on battery sleeps anyway.

7. Reference

FILE	WHAT
`companion/reader_companion/attach.py`	The attaching side: the disk (`Disk`), the FUSE operations, the WebDAV server, mounting with each system's own client, the daemon, attach codes, the driver installers.

FILE	WHAT
`companion/reader_companion/mount.py`	The sharing side: `share_folder`, `stop_share`, `list_shares`, and the socket's `list`, `rmtree`, `copy` and `statfs` ops for shares.
`companion/reader_companion/extension.py`	`ShareHandler` (`companion/share`) and `AttachHandler` (`companion/attach`).
`companion/reader_companion/cli.py`	`reader-companion share` and `reader-companion attach`.
`src/lib/shares.ts`, `src/components/SharedFolders.tsx`	The page: the calls, and the card in Settings → Compute & playgrounds.
`companion/tests/test_attach.py`	Tests: attach codes, the refusals, the commands each system mounts with, the disk end to end, WebDAV as Finder and Windows speak it, a real FUSE attach, and the two routes.

PART 2

How it was built, checked and measured

What it is and how to use it is in shared-folders.md. This document covers why it is built the way it is, how the pieces fit, what the tests found on the way, what is and isn't verified, and how to try it yourself.

1. The ask

> "Wherever the account is logged in via the app, I want to be able to mount a disk of one computer onto another ...
> flexible along Linux, macOS and Windows."

The project mount (Companion 0.19.0, compute-mount.md) already put a folder of your computer on a GPU machine. That GPU machine was a Linux box that the page reached through a Jupyter kernel. This feature is the general case: any two of your computers, any of three systems, either direction, and any folder.

2. The choices

Three questions were put to you before any code was written. Your answers set the design:

QUESTION	CHOSEN	WHAT IT MEANS IN THE CODE
What can be shared?	**Folders I pick**	`mount.share_folder(path)`: one key per folder. The whole disk and the home folder are refused.
How is it mounted?	**Built in by default, a driver offered**	WebDAV through the system's own client works with nothing installed. FUSE (FUSE-T, WinFsp, libfuse) is used when it is there, and **Install ...** fetches it.
How much is kept?	**Only what's used**	Lazy listing, 1 MB blocks fetched on read. `all` and `none` are options.

Three more choices were made along the way:

- **Reuse the mount socket, don't build a second one.** Shares ride `/companion/mount/socket` with the project mount's frames, its key checks and its path refusals. A share is a `Share(lazy=True)`: no tree, no project rules, and four new ops (`list`, `rmtree`, `copy`, `statfs`).
- **The page introduces, the Companions talk.** The page asks A for `{url, key}` and hands them to B. After that, B's daemon talks to A's Companion through A's tunnel. No file passes through the page, the Worker or Koios's servers.
- **A WebDAV server of its own, not a library.** About 300 lines of the standard library's `http.server` cover what Finder, Windows' WebClient and `gio` actually send: OPTIONS, PROPFIND, PROPPATCH, GET/HEAD with ranges, PUT (chunked too), DELETE, MKCOL, MOVE, COPY, and LOCK/UNLOCK, which all three clients insist on before they write. The Companion gains no dependency but fusepy, which is pure Python.

3. How it fits together

3.1 Attaching

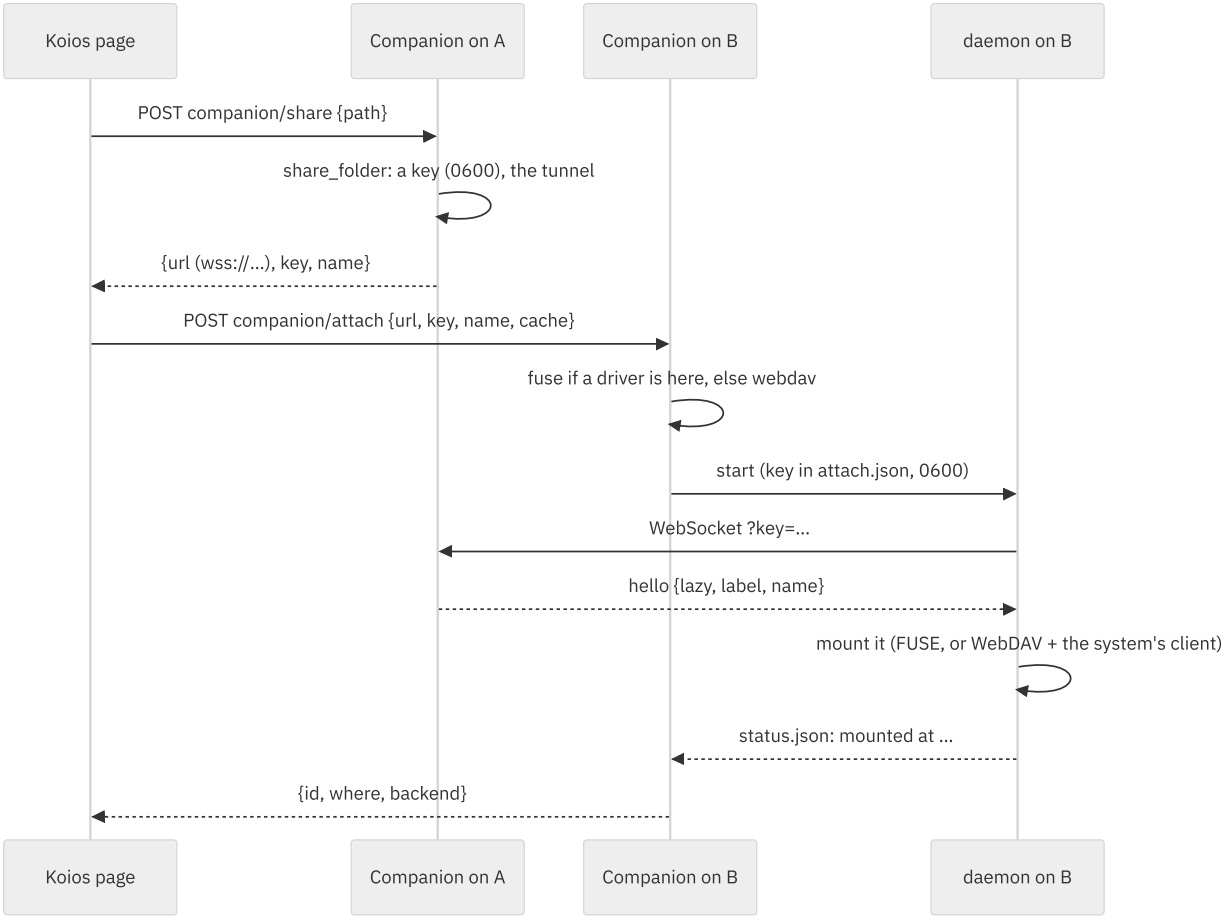


Figure 1. One call to each Companion. B answers once the disk is mounted, or with the daemon's own words if it couldn't be.

3.2 A read

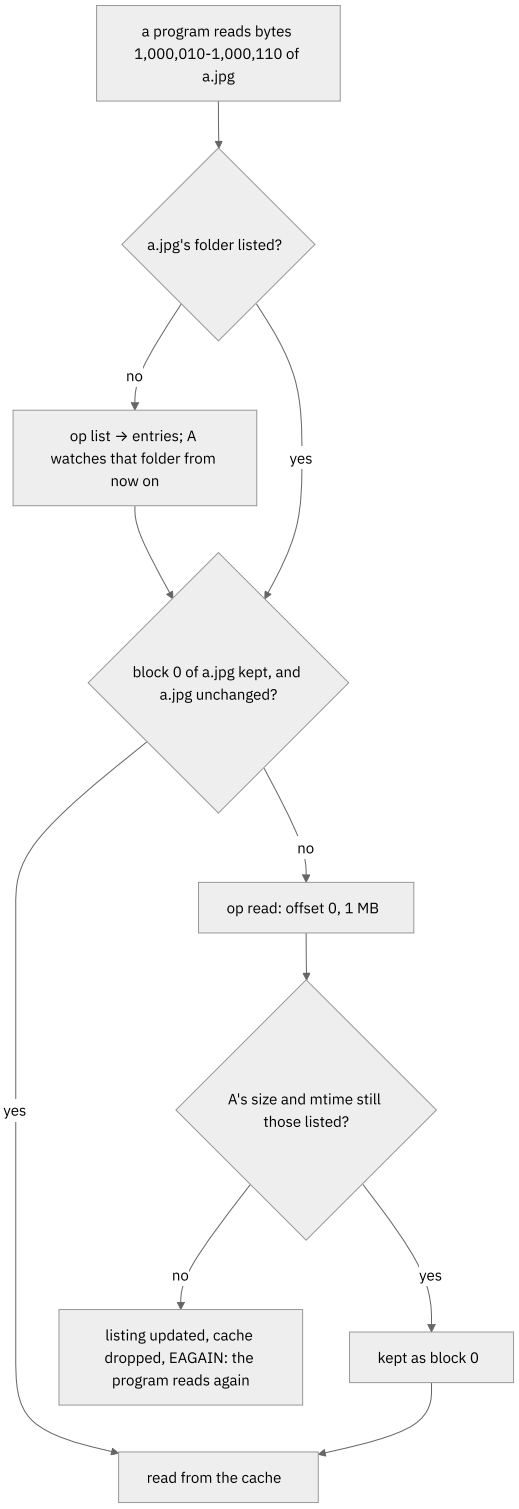


Figure 2. Only the blocks a read touches come over. A file that changes on A as it is read is never stitched from two versions.

3.3 A write, and a change on A

- **Here → A.** `open(…, "w")` writes into the cache copy and marks the file dirty. On `close()`, which the kernel turns into FUSE's `flush`, or when a WebDAV `PUT` ends, the file is sent in 4 MB pieces. A puts it

beside the original and swaps it in, then answers its new size and mtime, which the cache takes as the file's own.

- **A → here.** Twice a second, A looks at the folders that machines have listed (stat calls only). Each change goes to every machine as `{op: "entries", dir, changed, gone}`. The daemon updates its listing and drops a kept copy that is out of date.

3.4 Each system's way in

	FUSE	WEBDAV
Mac	fusepy + FUSE-T (`FUSE_LIBRARY_PATH` set to `libfuse-t.dylib` when it is there) or macFUSE; `volname` set, so Finder shows the name	`mount_webdav -S -v <name> http://127.0.0.1:<port>/<secret>/ <dir>`
Windows	fusepy + WinFsp; a free drive letter from Z: down	`net use * http://127.0.0.1:<port>/<secret>/ /persistent:no`; the letter is read from its answer
Linux	fusepy + libfuse 2	`gio mount dav://127.0.0.1:<port>/<secret>/`, linked from `~/Koios Drives/<name>`

Unmounting goes through each system's own tool: `fusermount -u`, `umount`, `diskutil unmount`, `net use /delete`, or `gio mount -u`. Detach asks the daemon to stop, waits 5 s, and then ends it.

---

4. The order of work, and what each run found

1. **The sharing side** (mount.py): `share_folder`, `record_for`, `Share(lazy)`, and the ops `list`, `rmtree`, `copy`, `statfs`. The project-mount tests (test_mount.py) still pass unchanged.
2. **The disk** (attach.py `Disk`): listing, block reads, writes, renames, and the background fill. The tests drove it in-process against a real socket, and they passed the first time except for one test-only mistake.
3. **WebDAV.** All of Finder's and Windows' verbs, sent with `http.client`, including a chunked PUT. These passed the first time.
4. **FUSE, for real.** `start()` spawns the daemon, which mounts a real FUSE filesystem, and the test writes through it. **Found:** a file written through the disk wasn't on A right after `close()`. The upload ran in FUSE's `release`, which the kernel sends *after* `close()` returns. **Fixed:** the upload runs in `flush`, which `close()` waits for, and `release` sends only what a later write left.
5. **The routes** (extension.py), driven by a test the way the page calls them. **Found:** a test-only deadlock. The FUSE read needed the socket, whose loop the test had paused. The read now runs from an executor.
6. **The whole suite. Found:** `stop_share` closed the sockets from whatever thread called it, and tornado wants its own loop for that. The Companion's handler calls it on the loop, but not every caller would. **Fixed:** each close is handed to its socket's loop with `add_callback`.

5. The tests

`companion/tests/test_attach.py`. On Linux in this sandbox all 11 pass, with /dev/fuse and libfuse 2, and the whole Companion suite passes too: 191 tests on Python 3.9, 3.13 and 3.14.

TEST	CHECKS
`Codes`	An attach code round-trips. The whole disk, the home folder and a missing folder are refused. The key is the same each time a folder is shared. The share file is 0600. Stopping a share forgets its key.
`MountCommands`	The exact `mount_webdav` and `net use` commands, and the drive letter read back from Windows' answer. Labels are cleaned of `/ : * " < > ` .
`Attached.test_only_what_is_opened…`	One listing, then one 1 MB block for a 100-byte read in the middle of a 3 MB file. Whole after a full read, and kept. A's changes are pushed, a write goes back, and mkdir, rename, copy, rmtree and statfs work.
`…stopped_share_cuts…`	Stop sharing closes the attached socket.
`…way_out_of_the_folder…`	`..` paths are refused.
`…cache_all…`, `…cache_none…`	The background fill gets every file. With `none`, nothing is kept.
`…webdav_the_way_finder…`	Wrong secret → 404. PROPFIND depth 0 and 1, GET with a Range and whole, HEAD, a chunked PUT, Finder's `._` file kept local, MKCOL (and 409 for a missing parent), MOVE, COPY with `Overwrite: F` → 412, LOCK/UNLOCK, DELETE of a folder.
`…attached_with_fuse_as_a_disk`	A real mount. Listing, a whole read, a write that is on A after close, `.DS_Store` kept local, rename, mkdir/rmdir, A's save seen here. Attaching again reuses the mount, and detach unmounts it.
`Routes`	403 without the token, for the page and for a local program. 400 for `/`. Share → code → listed without its key. The attach list and backends. A bad code → 400, a bad id → 400. Attach by code → the file is readable → detach → stop sharing.

6. What isn't verified yet

Plainly: the sandbox this was built in is Linux, so these weren't run here.

- **A real Mac or Windows mount.** FUSE-T, macFUSE and WinFsp were never loaded. On a Mac, `mount_webdav` against the Companion's WebDAV server, and on Windows, `net use`, are tested only as the commands built, not run. Finder and WebClient are known to be particular. Section 7 is the way to check them, and a quirk found there is a small fix in `DavHandler`.
- `gio` **on a GNOME desktop.** The sandbox has no session bus.
- **The driver installers.** `install_driver` downloads from FUSE-T's and WinFsp's GitHub releases. It was not run.
- **Across a real tunnel.** Every attach test used `ws://127.0.0.1`. The socket and frames are the project mount's, which were measured through a real Cloudflare tunnel: a save there in 0.24 s, a write back in

0.59 s (compute-mount-process.md §6.2). Expect a first read of an unkept block to cost one round trip through Cloudflare.

---

7. Trying it, step by step

7.1 One computer, both sides (10 minutes)

A Mac can attach its own folder. The bytes still go out through its tunnel to Cloudflare and back in.

1. Update the Companion: Settings → Updates → **Update**. It should say 0.20.0.
2. On the Mac:

```
   mkdir -p ~/Documents/koios-share-test && echo hello > ~/Documents/koios-share-
test/hello.txt
   reader-companion share ~/Documents/koios-share-test
   reader-companion attach koios-attach:…        # the code it printed
```

1. Finder: **~/Koios Drives/koios-share-test** shows `hello.txt`. Open it, edit it, save it, and the original in `~/Documents/koios-share-test` changes. Edit the original, and the attached copy changes within a second or two.
2. `reader-companion attach` lists it as `webdav` until a driver is installed. Then:

```
   reader-companion attach --install-driver      # FUSE-T's installer opens; finish it
   reader-companion attach --detach <id>
   reader-companion attach koios-attach:… --backend fuse
```

It should say `fuse` now. Try the same edits, and open a big file. With FUSE, only the part read comes over: `cachedFiles` in `~/.reader-companion/attached/<id>/status.json` grows only as files are opened whole.

7.2 Two computers (15 minutes)

1. Install the Koios app on the second computer (Windows, Linux or another Mac), sign in with the same account, and update its Companion to 0.20.0.
2. In Koios on either one: **Settings → Compute & playgrounds → Folders across your computers**. Pick From, the folder, To, then **Attach on …**.
3. Windows: the drive letter shows in File Explorer → This PC. Mac: `~/Koios Drives/<name>`. Linux: `~/Koios Drives/<name>`.
4. Sleep the computer with the folder, then open a file you haven't opened yet on the other: it waits. Wake it: the read finishes. With **Everything, in the background**, the same file opens while it sleeps.
5. **Stop sharing** on the first computer: the second loses the disk's connection at once.

7.3 If something goes wrong

- `~/.reader-companion/attached/<id>/attach.log` is the daemon's own log, and `status.json` its state (connected, cachedFiles, waitingToWrite, the last error).
- Windows: `net use` saying "the network name cannot be found" means the WebClient service is off. Run `sc config webclient start= auto` and `sc start webclient` (as administrator), or install

WinFsp.

- Linux with no GNOME: `sudo apt install libfuse2` and attach with `--backend fuse`.